

120 minutes. Tous Documents papier autorisés. Barème à titre indicatif.

1 Modélisation :

Le code de la figure Fig 1 présente un modèle très simple d'authentification.

1. Donner un schéma du système (processus, canaux, flux de l'information).
2. Ecrire en Promela un modèle du système.
3. On veut vérifier la propriété suivante :
 - *un client qui échoue à l'authentification ne reçoit pas de message du serveur.*Décrire formellement la vérification de cette propriété avec SPIN en utilisant un observateur.

2 Canaux Synchrones en Go :

On modélise le système suivant :

- des *clients* envoient des calculs (des fonctions *Go* de `unit` dans `int`) à un *serveur*,
 - côté client, l'envoi d'un calcul est géré comme une future abstraite, qui peut être consultée (*poll*) ou attendue (*wait*). Ces deux actions correspondent à l'envoi de messages au serveur sur des canaux différents.
 - côté serveur, les calculs de plusieurs clients sont gérés simultanément par des goroutines.
1. Donner un schéma du système (processus, canaux, flux de l'information).
 2. Ecrire le serveur en *Go*.
 3. Ecrire deux clients différents qui interagissent avec le serveur (avec deux calculs différents, au moins un des deux utilise le *polling*)
 4. Expliquer comment modifier le système pour permettre au client et au serveur de s'exécuter sur des machines différentes (en utilisant un protocole texte à définir).

3 Fair Threads :

On modélise le système suivant :

- un unique *producteur* produit des *paquets*, représentés par des entiers. Le paquet produit est stocké dans une boîte, et le producteur s'endort jusqu'à ce que la boîte soit vide (auquel cas il produit à nouveau).
 - les paquets sont collectés dans la boîte par trois *ouvriers* et apportés à trois piles (LIFO) de paquets (chaque ouvrier maintient sa propre pile).
 - régulièrement, un *sélectionneur* inspecte les têtes des trois piles, et dépile la tête ayant la valeur la plus grande,
 - quand une des trois piles a moins de trois éléments, le sélectionneur s'endort jusqu'à ce que toutes les piles contiennent au moins 3 éléments chacune.
1. Décrire à l'aide d'un schéma le *design* de ce modèle avec des *Fair Threads*, en prenant soin de justifier le nombre d'ordonnanceurs utilisés, et leur rôle.

2. Donner l'implémentation FT de ce modèle.

4 Canaux Synchrones en OCaml:

Ecrire un programme OCaml avec le module **Event** modélisant le processus P suivant :

- à l'initialisation, on fournit à P une liste de couples $(c_0, s_0), \dots, (c_n, s_n)$, les c_i sont des canaux et les s_i des chaînes de caractères, on appelle s_i le "propriétaire" de c_i
 - P écoute simultanément sur tous les c_i et sur un canal supplémentaire s .
 - P reçoit, en boucle, des entiers sur les c_i (il peut recevoir plusieurs entiers d'un même c_i , l'ordre des réceptions est libre - par exemple, il peut recevoir deux fois sur c_3 puis une fois sur $c_1, \dots$).
 - quand P reçoit $()$ sur s , il arrête d'écouter et envoie sur un canal o une chaîne de caractères correspondant au propriétaire du canal sur lequel a été reçue la plus grande valeur entière.
1. Donner un schéma rapide du système (canaux, flux de l'information)
 2. Donner le code de P .

```

package main

import (
)

type auth struct {
    login string
    mdp string
    contact chan string
}

func client(l string, m string, url chan auth) {
    c := make(chan string)
    url <- auth{login : l, mdp : m, contact : c}
    for {
        reponse := <- c
        println("Client", l, "recoit", reponse, "\n")
    }
}

func authentifieur(url chan auth, s chan chan string){
    noms := [5]string{"Alice", "Bob", "Carole"}
    mdps := [5]string{"123", "mdp", "234"}
    for{
        a := <- url
        trouve := false
        for i := 0; i < len(noms) && !trouve; i++){
            if (noms[i] == a.login) && (mdps[i] == a.mdp){
                a.contact <- "OK"
                s <- a.contact
                trouve = true
            }
        }
        if !trouve {a.contact <- "KO"}
    }
}

func serveur(s chan chan string){
    for{
        c := <- s
        c <- "BIENVENUE"
    }
}

func main(){
    noms := [5]string{"Alice", "Bob", "Carole", "David", "Elena"}
    mdps := [5]string{"123", "mdp", "123", "chaussette", "123"}
    url := make(chan auth)
    s := make(chan chan string)
    go func(){authentifieur(url, s)} ()
    go func(){serveur(s)}()
    for i := 0 ; i < 5 ; i++ {
        go func(k int){client(noms[k], mdps[k], url)}(i)
    }
    for{
    }
}

```

Figure 1: Authentification en Go.